

ALLTAGSENGEL

Review: Verordnungs-Workflow Alltagsengel

Datum: 31.07.2026 **Geprüfte Dateien:** - `supabase/migrations/20260719_eylem_audit_complete_features.sql` - `supabase/migrations/20260730_verordnungen_workflow_complete.sql` - `supabase/migrations/20260731_verordnungen_erweiterung.sql` - `app/admin/verordnungen/page.tsx` (1862 Zeilen) - `app/admin/leistungspreise/page.tsx` - `app/admin/kostentraeger/page.tsx` - `lib/admin/ops.ts`, `components/admin/OpsUI.tsx`

Gesamturteil: Der Workflow (Erfassen → Genehmigung → Verplanung → Abrechnung → Absagen) ist als **internes Verwaltungstool solide gebaut** und deutlich besser als eine Excel-Liste. Er ist aber **kein abrechnungsfähiges System gegenüber Kassen** — es fehlen Pflegegrad, Geburtsdatum, DTA/§302, Vergütungsvereinbarungs-Bezug und die Abtretungserklärung. Zusätzlich gibt es **eine echte Sicherheitslücke (RLS)** und **einen fachlichen Konstruktionsfehler** (das Modul modelliert §37-SGB-V-Behandlungspflege, obwohl Alltagsengel ein §45a-Betreuungsdienst ist).

1. KRITISCH (sofort beheben)

1.1 RLS-Policies der neuen Tabellen sind faktisch offen ●

`20260731_verordnungen_erweiterung.sql` legt drei Tabellen mit dieser Policy an:

```
CREATE POLICY "leistungspreise_admin_all" ON leistungspreise
  FOR ALL USING (true) WITH CHECK (true);
-- identisch für kostentraeger_kontakte und einsatz_absagen
```

`USING (true)` heißt: **jeder Request mit gültigem JWT** (jeder eingeloggte Kunde, jeder Engel, je nach Grants sogar `anon`) darf lesen, schreiben **und löschen**. Der Policy-Name sagt „admin_all“, die Policy prüft aber keinerlei Rolle. `einsatz_absagen` enthält personenbezogene Daten (wer hat abgesagt, Grund — potenziell Gesundheitsdaten wie „Klientin im Krankenhaus“). Das ist ein DSGVO-Problem und ein Datenintegritäts-Problem (Kunde könnte Preisliste manipulieren).

Alle anderen Migrationen im Projekt machen es richtig (`public.is_admin()`). **Fix:**

```
DROP POLICY IF EXISTS "leistungspreise_admin_all" ON leistungspreise;
CREATE POLICY "leistungspreise_admin_all" ON leistungspreise
  FOR ALL USING (public.is_admin()) WITH CHECK (public.is_admin());
CREATE POLICY "leistungspreise_service_all" ON leistungspreise
  FOR ALL TO service_role USING (true) WITH CHECK (true);
-- analog für kostentraeger_kontakte und einsatz_absagen
```

1.2 Fachlicher Konstruktionsfehler: §37-Logik in einer §45a-App ●

Alltagsengel ist **Alltagsbegleitung nach §45a SGB XI** — kein zugelassener Pflegedienst. Trotzdem:

- Das Formular hat als **Default** `verordnung_type: 'behandlungspflege_37'` (`page.tsx` Zeile 116). Behandlungspflege §37 SGB V darf nur ein Pflegedienst mit Versorgungsvertrag nach **§132a SGB V** erbringen und abrechnen. Gleiches gilt für Pflegesachleistung §36 (`haeusliche_pflege_36`) — die setzt einen Versorgungsvertrag nach **§72 SGB XI** voraus.
- Die `leistungsart`-Liste (große/kleine Körperpflege, Hilfe beim Ausscheiden, Behandlungspflege) sind die **Grundpflege-Leistungskomplexe der häuslichen Krankenpflege** — nicht das Leistungsspektrum eines Betreuungsdienstes.
- Der Formular-Hinweis „Gilt für Pflegedienst + Betreuung, NICHT Intensivpflege“ verstärkt die Verwirrung.

Konsequenz in der Praxis: Wenn das Büro-Team hiermit arbeitet, erfasst es Verordnungen für Leistungen, die Alltagsengel gar nicht abrechnen darf. Entweder gehört diese Logik in **efy care** (§37 SGB V, dort ist sie richtig), oder das Modul muss auf das echte Alltagsengel-Spektrum umgestellt werden:

Rechtsgrundlage	Realität	Braucht ärztliche Verordnung?	Braucht Kassengenehmigung?
§45b Entlastungsbetrag (131 €/Monat)	Kerngeschäft	Nein	Nein — nur Anerkennung des Anbieters nach Landesrecht
§39 Verhinderungspflege (seit 07/2025: gemeinsamer Jahresbetrag mit Kurzzeitpflege)	häufig	Nein	Antrag/Anzeige bei der Kasse, keine „Verordnung“
§36 Sachleistung (Betreuungs-/Entlastungsleistungen über Umwandlungsanspruch §45a Abs. 4)	möglich	Nein	Umwandlungserklärung des Klienten
§37 SGB V Behandlungspflege	nicht zulässig für Alltagsengel	Ja (Muster 12)	Ja

Das Datenmodell vermischt zwei Konzepte: „ärztliche Verordnung mit Kassengenehmigung“ (§37-Welt) und „Budget-Anspruch/Bewilligungsbescheid“ (§45b/§39-Welt). Beim Entlastungsbetrag gibt es keinen Arzt, keine Verordnungsnummer, kein Muster 12 — dafür braucht es die **Abtretungserklärung** (siehe 1.4). Empfehlung: `verordnung_type` auf `bewilligung_45b`, `verhinderung_39`, `umwandlung_45a4`, `privat`, `sonstige` einschränken (oder ein `basis`-Feld `verordnung | bewilligung | erklaerung` einführen) und die §37-Werte nach efy care verschieben.

1.3 Pflegegrad und Geburtsdatum des Klienten fehlen komplett ●

`clients` hat (nach 20260719) `versicherтенnummer`, `pflegekasse_name`, `pflegekasse_ik` — **aber keinen Pflegegrad und kein Geburtsdatum**. Nur `care_recipients` (Marktplatz) hat `pflegegrad`. Ohne diese beiden Felder ist **kein einziges Kassenformular ausfüllbar**: Jeder Leistungsnachweis, jede Rechnung an die Pflegekasse und jeder §45b-Erstattungsantrag verlangt Name, Geburtsdatum, Versichertennummer und Pflegegrad. Zudem: §45b setzt **mindestens Pflegegrad 1 voraus** — ohne das Feld kann die App nicht einmal validieren, ob ein Klient anspruchsberechtigt ist.

```
ALTER TABLE public.clients
  ADD COLUMN IF NOT EXISTS geburtsdatum date,
  ADD COLUMN IF NOT EXISTS pflegegrad integer CHECK (pflegegrad BETWEEN 1 AND 5),
  ADD COLUMN IF NOT EXISTS pflegegrad_seit date;
```

ICD-10: `diagnose` ist Freitext. Für §45a genügt das; falls das Modul für efy care/§37 weiterverwendet wird, braucht es `icd10_codes text[]` (Muster 12 verlangt ICD-10).

1.4 Abtretungserklärung fehlt — das zentrale Abrechnungsdokument im §45b-Geschäft ●

In der Praxis läuft §45b-Abrechnung fast immer so: Der Klient unterschreibt eine **Abtretungserklärung**, der Dienst rechnet direkt mit der Pflegekasse ab (Kostenerstattung an den Dienst statt an den Klienten). Ohne dokumentierte Abtretung zahlt die Kasse nicht an Alltagsengel. Es gibt dafür **kein Feld, keine Tabelle, keinen Upload**. Vorschlag:

```
ALTER TABLE public.clients
  ADD COLUMN IF NOT EXISTS abtretung_vorhanden boolean NOT NULL DEFAULT false,
  ADD COLUMN IF NOT EXISTS abtretung_datum date,
  ADD COLUMN IF NOT EXISTS abtretung_document_url text; -- privater Bucket wie verordnungen
```

Plus Warnung in der Abrechnungs-UI: „Rechnung an Kasse ohne Abtretungserklärung → wird nicht bezahlt.“

1.5 Verordnungen sind hart löschar — Verstoß gegen Revisionssicherheit ●

`remove()` macht `DELETE` auf `verordnungen`; die FKs von `invoices` / `service_records` stehen auf `ON DELETE SET NULL`. Folge: Eine Verordnung mit **abgerechneten Leistungen** kann gelöscht werden, die Rechnungen verlieren stillschweigend ihren Genehmigungsbezug. Bei einer Kassen- oder Wirtschaftsprüfung (GoBD, Aufbewahrungspflicht 10 Jahre für Abrechnungsunterlagen) ist das nicht haltbar. Außerdem bleibt der Scan als Leiche im Storage-Bucket.

Fix: Soft-Delete (`archived_at`) statt `DELETE`; Löschen blockieren, wenn verknüpfte `invoices` oder `service_records` existieren:

```
const { count } = await supabase.from('invoices')
  .select('id', { count: 'exact', head: true }).eq('verordnung_id', id)
if (count && count > 0) {
  setError('Verordnung hat verknüpfte Rechnungen – Archivieren statt Löschen.')
  return
}
```

2. WICHTIG (zeitnah beheben)

2.1 Doppelte Preistabellen: `service_pricing` VS. `leistungspreise`

20260719 legt `service_pricing` an (Quelle der Wahrheit, Seed-Daten, saubere RLS), 20260731 legt **parallel** `leistungspreise` an (Bundesland × Leistungsart). Zwei nicht verknüpfte Preissysteme mit unterschiedlichen Leistungsart-Taxonomien (`SERVICE_TYPES` vs. `LEISTUNGSART_LABELS`). Das wird auseinanderlaufen. Zudem ist das Modell „Preis je Bundesland“ fachlich verkürzt: real werden Preise je **Vergütungsvereinbarung mit dem Kostenträger** festgelegt (§45b landesrechtlich, teils je Kasse gedeckelt). `leistungspreise` sollte optional `kostentraeger_kontakt_id` referenzieren und eine der beiden Tabellen sollte die andere ablösen.

2.2 Keinerlei Automatisierung — die Erinnerungs-Flags sind tote Spalten

`erinnerung_30_tage/14/7` existieren seit 20260719 in der DB, aber **nichts setzt sie und nichts verschickt etwas** (kein `pg_cron`, keine Edge Function, kein Task). Die Ablauf-Ampel funktioniert nur, wenn jemand die Seite öffnet. In der Praxis zwingend:

- **Cron-Job (täglich):** Genehmigungen mit `genehmigung_bis` in $\leq 30/14/7$ Tagen → Flag setzen + Benachrichtigung (Notifications-System existiert bereits: `20260316_notifications.sql`) + optional E-Mail-Entwurf an die Arztpraxis (Freigabe durch Yusuf gemäß Regel „Erst zeigen, dann senden“).
- **Auto-Statuswechsel:** `genehmigung_status = 'abgelaufen'` wenn `genehmigung_bis < today` — aktuell bleibt der Status ewig „genehmigt“, die Verordnung erscheint weiter in Verplanung/Abrechnung als aktiv.
- **Auto-`abrechnungs_status`:** wird heute rein manuell per Dropdown gepflegt und kann von der Realität abweichen; aus verknüpften Rechnungen ableitbar.

2.3 Widerspruchsverfahren: nur ein Status, keine Fristen

Es gibt `genehmigung_status = 'widerspruch'`, aber: kein Bescheid-Datum, kein Widerspruchs-Datum, keine Fristüberwachung. Die **Widerspruchsfrist beträgt 1 Monat ab Bescheid** — verpasst = Geld weg. Minimal:

```
ALTER TABLE verordnungen
  ADD COLUMN IF NOT EXISTS ablehnung_bescheid_datum date,
  ADD COLUMN IF NOT EXISTS widerspruch_frist_bis date,    -- Bescheid + 1 Monat
  ADD COLUMN IF NOT EXISTS widerspruch_eingelegt_am date,
  ADD COLUMN IF NOT EXISTS widerspruch_ergebnis text
  CHECK (widerspruch_ergebnis IN ('stattgegeben','teilweise','zurueckgewiesen'));
```

Plus rote Ampel „Widerspruchsfrist läuft in X Tagen ab“ bei abgelehnten Verordnungen.

2.4 Genehmigungsfiktion §13 Abs. 3a SGB V wird nicht abgebildet

Die Wartezeit-Anzeige („>21 Tage — nachhaken!“) ist gut, aber unvollständig: Entscheidet die Kasse nicht binnen **3 Wochen** (5 mit Gutachten), gilt die Leistung als genehmigt (Genehmigungsfiktion). Statt nur „nachhaken“ sollte das UI nach 21 Tagen anbieten: „Genehmigungsfiktion dokumentieren“ (Status genehmigt + Vermerk). Das ist bares Geld. (Relevant v. a. für den §37-Teil / efy care.)

2.5 Erfassungsformular umgeht den eigenen Workflow

Im Formular (Tab 1) kann man `genehmigung_status` frei auf „Genehmigt“ setzen — dann fehlen `kassengenehmigung_beantragt_am / antwort_am`, die Pflicht-Genehmigungsnummer wird nicht erzwungen (Tab 2 erzwingt sie), und die Dauer-Statistik in „Entschieden“ zeigt Unsinn. Entweder Status-Feld aus dem Erfassungsformular entfernen (Workflow nur über Tab 2) oder beim direkten Setzen auf „genehmigt“ dieselben Validierungen anwenden.

2.6 Fehlende Validierungen

- `gueltig_von <= gueltig_bis` wird nirgends geprüft (weder UI noch DB). `CHECK (gueltig_bis IS NULL OR gueltig_von IS NULL OR gueltig_von <= gueltig_bis)`.
- IK-Nummer: Freitext ohne Format-Check. IK ist **9-stellig numerisch** — `pattern / CHECK (~ '^\d{9}$')`.
- `ausstellungsdatum` in der Zukunft möglich.
- Versichertennummer-Format (1 Buchstabe + 9 Ziffern) ungeprüft.
- `leistungspreise`: UNIQUE auf `(bundesland, leistungsart, gueltig_ab)` verhindert keine **überlappenden Gültigkeitszeiträume** — `findLeistungspreis` nimmt dann still den neuesten.

2.7 Verplanung: keine Kontingent- und Kollisionsprüfung

`saveAssign()` legt Einsätze an ohne zu prüfen: (a) ob der Engel am selben Wochentag/Zeitfenster schon verplant ist (Doppelbuchung), (b) ob die Summe der geplanten Wochenstunden `genehmigte_stunden_pro_woche` übersteigt (= unbezahlte Arbeit, genau das, was der Seitenkommentar verhindern will), (c) `start_time < end_time`. Mindestens (b) sollte als Warnung erscheinen — die Daten sind alle schon im State vorhanden.

2.8 `approvedHours()` überschätzt das Kontingent

```
const weeks = Math.max(1, Math.ceil(ms / (7 * 86400000)))
```

`Math.ceil` zählt eine angebrochene Woche voll (Zeitraum 01.01.–15.02. = 6,4 Wochen → 7). Bei 5 h/Woche sind das 3,5 h zu viel — der Verbrauchsbalken zeigt zu spät Rot, und man arbeitet über das Genehmigte hinaus. Korrekt wäre anteilig: `(ms / (7*86400000))` ohne `ceil` (bzw. kaufmännisch je nach Kassenlogik) — im Zweifel **abrunden**, nie aufrunden.

2.9 `einsatz_absagen` erfasst nicht, WELCHER Termin abgesagt wurde

Assignments sind wiederkehrend (Wochentag). Die Absage speichert nur `abgesagt_am` (Zeitpunkt der Absage-Meldung), nicht das **Datum des betroffenen Einsatzes**. „Klientin sagt am Montag den Einsatz von Donnerstag ab“ ist nicht abbildbar; der Datumsfilter im UI filtert nach Meldedatum. Fehlt: `einsatz_datum date NOT NULL`. Außerdem fehlt der Index auf die FK: `CREATE INDEX ON einsatz_absagen(assignment_id);` (CASCADE-FK ohne Index = Seq-Scan bei jedem Assignment-Delete).

2.10 Kein Audit-Trail auf `verordnungen`

Wer hat den Genehmigungsstatus geändert? Wer hat die Kürzung erfasst? Bei Kassenstreitigkeiten braucht man das. Minimal: `updated_by uuid` + History-Tabelle oder Trigger-basiertes `verordnungen_audit_log`.

3. ABRECHNUNGS- & BRANCHENSTANDARD-LÜCKEN

3.1 Kein DTA nach §302 SGB V / §105 SGB XI — größte strategische Lücke

Es gibt **keinerlei** maschinenlesbare Datenübermittlung: kein DTA-Export (Nutzdaten-/Auftragsdatei nach den §302-Richtlinien der GKV), keine Anbindung an ein Abrechnungszentrum (DMRZ, Optica, Noventi, azh). Konsequenzen:

- Kassen dürfen bei nicht-maschinellem Abrechnung die Vergütung **um bis zu 5 % kürzen** (§302 Abs. 2 SGB V; analog §105 Abs. 2 SGB XI für Pflegeleistungen). Das `kuerzung_cent`-Feld wird also regelmäßig gefüllt werden — die App dokumentiert das Symptom, statt die Ursache zu lösen.
- Papier-/PDF-Rechnungen verlängern die Zahlungsziele massiv.

Pragmatische Empfehlung für die Firmengröße: Nicht selbst DTA (EDIFACT PLGA/PLAA, Verschlüsselung, ITSG-Trust-Center-Zertifikat) implementieren — das ist ein Projekt für sich. Stattdessen **CSV/API-Export für ein Abrechnungszentrum** bauen (DMRZ hat eine JSON-API und nimmt auch §45b-Betreuungsleistungen an). Das Feld `elektronisch_abrechenbar` auf `kostentraeger_kontakte` deutet an, dass das Thema bekannt ist — es gibt aber keinen Export-Button. Konkret: Route `/api/admin/abrechnung/dta-export` die je Monat/Kostenträger die `service_records` + `invoices` als DMRZ-kompatible Datei ausgibt.

3.2 Kein kassenkonformes Leistungsnachweis-Template je Verordnung

Es existiert `app/api/leistungsnachweis/route.ts` (PDF, gut!), aber der Verordnungs-Tab 4 zeigt nur eine Tabelle. Kassen verlangen je Abrechnung einen Leistungsnachweis mit: Versichertennummer, Pflegegrad (fehlen — siehe 1.3), Genehmigungs-Aktenzeichen, Einzeleinsätze mit Datum/Uhrzeit von-bis, **Unterschrift des Versicherten je Einsatz** (Signature-API existiert bereits — muss nur in das PDF je Verordnung einfließen) und Unterschrift/Stempel des Dienstes mit IK. Fehlend: Button „Leistungsnachweis für Kasse erzeugen“ in Tab 4, der das bestehende PDF mit Aktenzeichen + IK (460629986) + DejaVuSans generiert.

3.3 Vergütungsvereinbarungen nicht modelliert

Preise hängen real an der Vereinbarung mit dem Kostenträger (inkl. Gültigkeit, Kilometerpauschale, Wochenend-/Feiertagszuschlag). `leistungspreise` kennt weder Kostenträger noch Zuschläge noch Fahrtkosten (`BILLING_UNIT` kennt immerhin `kilometer`, aber die Preistabelle nutzt es nicht).

3.4 Tourplanung / Routenoptimierung fehlt

Verplanung ist eine flache Wochentag-Liste ohne Geo-Bezug. Für die aktuelle Größe verkraftbar (nice-to-have), aber: es gibt bereits `geo-events` (`app/api/native/geo-events/route.ts`) — eine einfache Tagesansicht „welcher Engel ist wann wo“ mit Fahrzeit-Warnung bei Überschneidungen wäre mit vorhandenen Daten machbar. Volle Routenoptimierung ist Overkill.

3.5 Eilgenehmigung / vorläufige Leistungserbringung

Kein Konzept für dringende Fälle (Krankenhausentlassung Freitag, Betreuung ab Montag nötig). Praxis: Leistungsbeginn vor Genehmigung mit Risiko-Vermerk. Minimal: Flag `eilfall boolean` + Banner „Leistung läuft ohne Genehmigung — Kostenrisiko“.

4. PERFORMANCE & CODE-QUALITÄT

- **Indizes:** Die wichtigen sind da (`gueltig_bis`, `abrechnungs_status`, FK-Indizes auf den Verknüpfungen). Fehlend: `einsatz_absagen(assignment_id)` (siehe 2.9), `verordnungen(client_id, genehmigung_status)` als Verbund für die häufigste Filterung.
- **`load()` lädt alles:** 8 parallele Queries ohne Pagination — alle Verordnungen, alle Assignments, alle Absagen. Bei <100 Klienten okay; ab ein paar hundert Verordnungen wird die Seite zäh. Kein Blocker heute, aber bewusst so lassen und bei Wachstum paginieren.
- **`daysUntil()` für Wartezeit missbraucht** (Zeile 1217: `-(daysUntil(...))` auf einem timestamp) — funktioniert, ist aber fragil; eine `daysSince()`-Helper wäre klarer.
- **1862-Zeilen-Client-Component:** Alle 5 Tabs in einer Datei, alles Client-seitig via Supabase-Browser-Client. Funktioniert dank RLS, aber gegen das eigene Next.js-Pattern (Server/Client-Split). Aufteilen in `components/admin/verordnungen/*Tab.tsx` würde Wartbarkeit deutlich erhöhen.
- **Storage-Leiche beim Scan-Ersatz:** Beim Hochladen eines neuen Scans wird der alte Pfad überschrieben, die alte Datei aber nicht gelöscht (bzw. bei Verordnungs-Delete bleibt der Scan liegen). DSGVO-Löschkonzept beachten.

5. WAS GUT GELÖST IST ✓

Damit klar ist, was funktioniert und nicht angefasst werden muss:

1. **Der 5-Schritte-Workflow als Tab-Struktur** ist genau richtig gedacht: Erfassen → Genehmigung → Verplanung → Abrechnung → Absagen spiegelt den echten Büro-Ablauf. Die Zähler auf den Tabs („3 unverplant“) sind gutes Aufgaben-Management.
2. **Privater Storage-Bucket mit signierten URLs** für Verordnungs-Scans — DSGVO-korrekt gelöst, inkl. Umlaut-sicherem `sanitizeFileName`.
3. **Abgleich beantragte vs. genehmigte Leistungsart** (`genehmigung_abgleich_ok` + Abweichungs-Banner) — das ist ein Detail, das viele kommerzielle Systeme nicht haben, und in der Praxis Gold wert (Kassen genehmigen gern etwas anderes als beantragt).
4. **SOLL/IST/Kürzung-Tracking auf Rechnungen** mit Kürzungsgrund — realistische Abbildung dessen, was Kassen tatsächlich tun.
5. **Wartezeit-Tracking** („beantragt vor X Tagen — nachhaken!“) und die Dauer-Statistik Antrag→Antwort.
6. **Ampel-Logik 30/14 Tage** mit Zeilen-Hervorhebung — die Kernfunktion „kein Klient ohne gültige Genehmigung“ ist im UI präsent.
7. **Verknüpfung** `service_records` / `assignments` / `invoices` ↔ **Verordnung** mit korrekten Indizes — die Datenbasis für „Verbrauch je Genehmigungsnummer“ stimmt, der Fortschrittsbalken Ist/ Genehmigt ist genau das, was die PDL braucht.
8. **Idempotente Migrationen** (`IF NOT EXISTS` , `text+CHECK` statt Enums, `DO $$ ... duplicate_object`) — sauberes Muster, konsistent mit dem Rest des Projekts.
9. **Pflicht-Gennehmigungsnummer bei Status „genehmigt“** in Tab 2.
10. **RLS auf** `verordnungen` **selbst ist korrekt** (`admin_all` / `staff_read` / `service_role` / `client_read` auf eigene) — die Lücke betrifft nur die drei Tabellen aus 20260731.
11. **Kostenträger-Kontaktdatenbank** inkl. Fax (ja, Kassen faxen noch) und `elektronisch_abrechenbar`-Flag.

6. PRIORISIERTE MASSNAHMENLISTE

#	Maßnahme	Aufwand	Priorität
1	RLS-Fix <code>leistungspreise</code> / <code>kostentraeger_kontakte</code> / <code>einsatz_absagen</code> → <code>is_admin()</code>	30 min	● sofort
2	<code>clients</code> : <code>pflegegrad</code> , <code>geburtsdatum</code> + im Verordnungs-/ Rechnungskontext anzeigen	2 h	● sofort
3	Verordnungstypen auf §45a-Realität umstellen (bzw. §37-Teil zu efy care), Default nicht <code>behandlungspflege_37</code>	3 h	● diese Woche
4	Abtretungserklärung (Felder + Upload + Warnung in Abrechnung)	3 h	● diese Woche
5	Hartes Löschen blockieren bei verknüpften Rechnungen/Records, Soft-Delete	2 h	● diese Woche

#	Maßnahme	Aufwand	Priorität
6	Cron: Ablauf-Erinnerungen + Auto-Status <code>abgelaufen</code>	4 h	●
7	Widerspruchs-Fristen (Felder + Ampel)	2 h	●
8	Validierungen (Datumslogik, IK 9-stellig, Kontingent-Warnung bei Verplanung)	3 h	●
9	<code>einsatz_absagen.einsatz_datum</code> + FK-Index	1 h	●
10	Preistabellen konsolidieren (<code>service_pricing</code> vs. <code>leistungspreise</code>)	4 h	●
11	Leistungsnachweis-PDF je Verordnung (Aktenzeichen, IK, Unterschriften, DejaVuSans)	1 Tag	●
12	Export an Abrechnungszentrum (DMRZ-API/CSV) statt eigenem DTA	2-3 Tage	● strategisch
13	Genehmigungsfiktion §13 Abs. 3a dokumentierbar machen	2 h	●
14	Tagesansicht Verplanung mit Geo/Fahrzeit (Tourplanung light)	2-3 Tage	● nice-to-have